

GQA

Grouped-Query Attention

Training Generalized Multi-Query Transformer Models
from Multi-Head Checkpoints

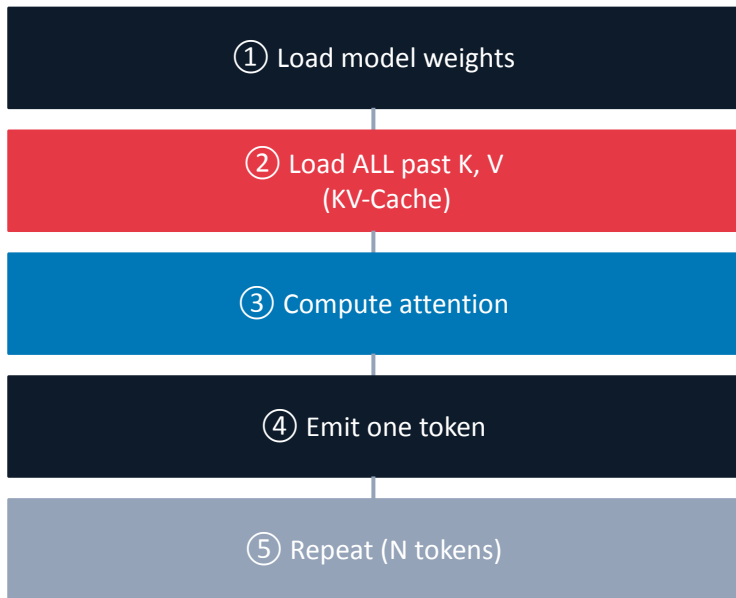
Ainslie, Lee-Thorp, de Jong et al. — Google Research (2023)

Presented by: Sanskriti Goyal

Why Is LLM Inference So Slow?

1 / 12

The Autoregressive Loop



● The Real Bottleneck

Each decode step must reload the entire KV-cache from HBM memory — NOT compute-bound, MEMORY-BANDWIDTH-bound.

KV-Cache Size

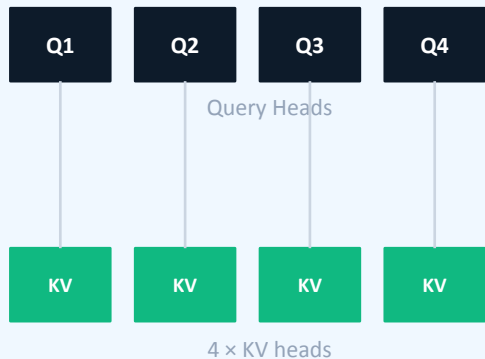
$$2 \times H \times d_k \times \text{seq_len} \times \text{bytes}$$

For T5-XXL: $H=64$ heads, $d_k=64$
→ hundreds of MBs per sequence
→ reloaded every single step

Three Attention Architectures

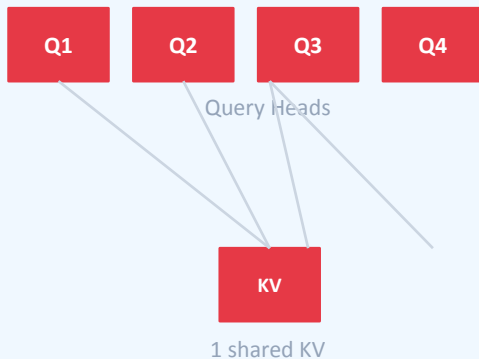
2 / 12

**Multi-Head
(MHA)**



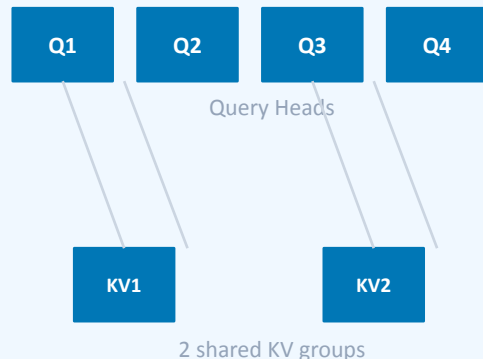
Cache: $H \times d_k$ per step

**Multi-Query
(MQA)**



Cache: $1 \times d_k$ per step ✓

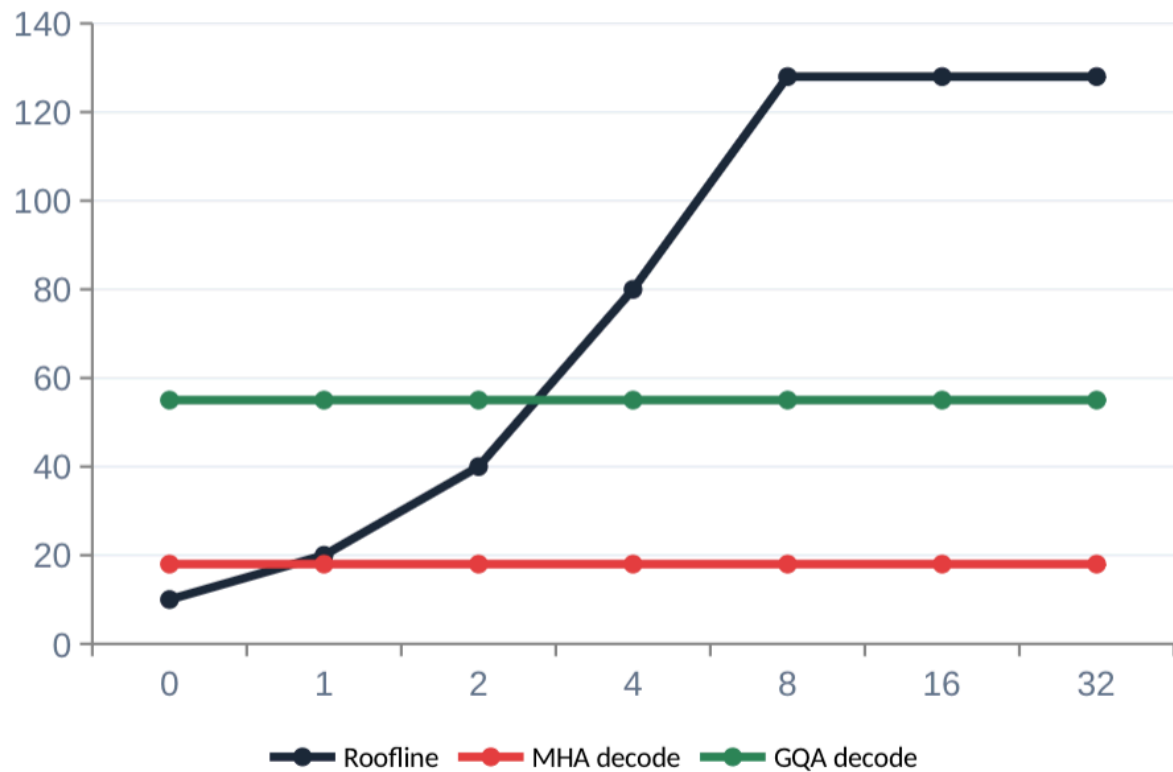
**Grouped-Query
(GQA, G=2)**



Cache: $G \times d_k$ per step ✓

MHA → quality ✓, speed ✗ **MQA** → speed ✓, quality ✗ **GQA** → the sweet spot: quality ≈ MHA, speed ≈ MQA

The Roofline Model: where decode lives



MHA decode: memory-bound
→ spends most time loading KV cache

GQA decode: moves toward
compute-bound region

Reducing KV heads → less data to load → higher arithmetic intensity

Walkthrough: "The cat sat" — MHA vs MQA vs GQA

3 / 12

Sentence: "The cat sat on the mat" → 4 query heads, step: decoding "on"

MHA

The

cat

sat

on

mat

Q1 → own K1,V1

Q1 attends via its private K1,V1

Q2 → own K2,V2

Q2 attends via its private K2,V2

Q3 → own K3,V3

Different projection per head

Q4 → own K4,V4

Cache stores 4 KV pairs

MQA

The

cat

sat

on

mat

Q1 → shared KV

All queries use same K,V

Q2 → shared KV

Single K,V projection

Q3 → shared KV

4× smaller KV cache

Q4 → shared KV

But capacity ↓↓

GQA (G=2)

The

cat

sat

on

mat

Q1,Q2 → KV-Group 1

Q1 & Q2 share K1,V1

Q3,Q4 → KV-Group 2

Q3 & Q4 share K2,V2

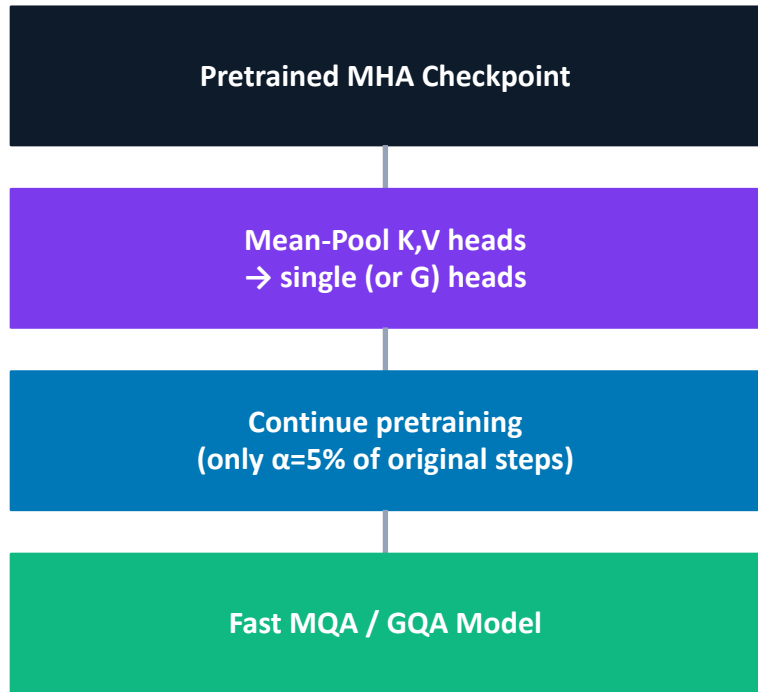
2 KV pairs cached

2× smaller than MHA

Capacity > MQA

Groups can specialize

Conversion Pipeline



Why Mean-Pool? (Fig 4)

Mean: 54.8 ✓ best

First: 54.6 (drops info)

Random: 54.4 (forgets all)

Preserving information from all heads matters — mean pooling retains the most signal.

Cost: Just 5% of Pretraining

≈ 600 TPUv3 chip-days vs 12,000+ for full training. You get a new, faster model for almost free.

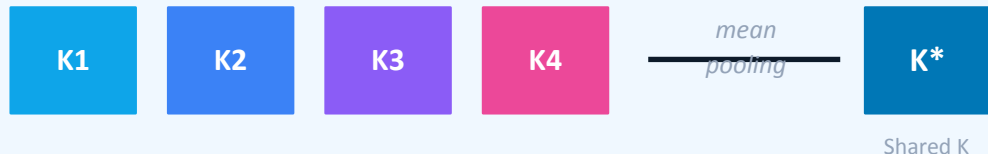


Discussion: Is 5% the right amount? Could companies skip uptraining entirely?

From MHA to MQA: Head Pooling (Fig 1)

5 / 12

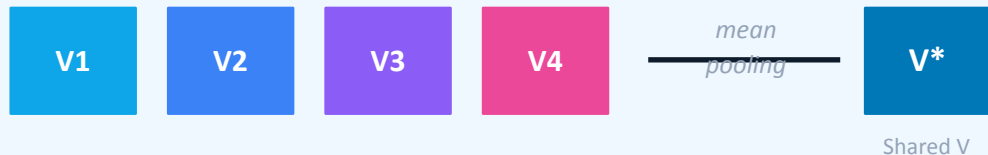
K heads (before)



Q heads (unchanged)



V heads (before)

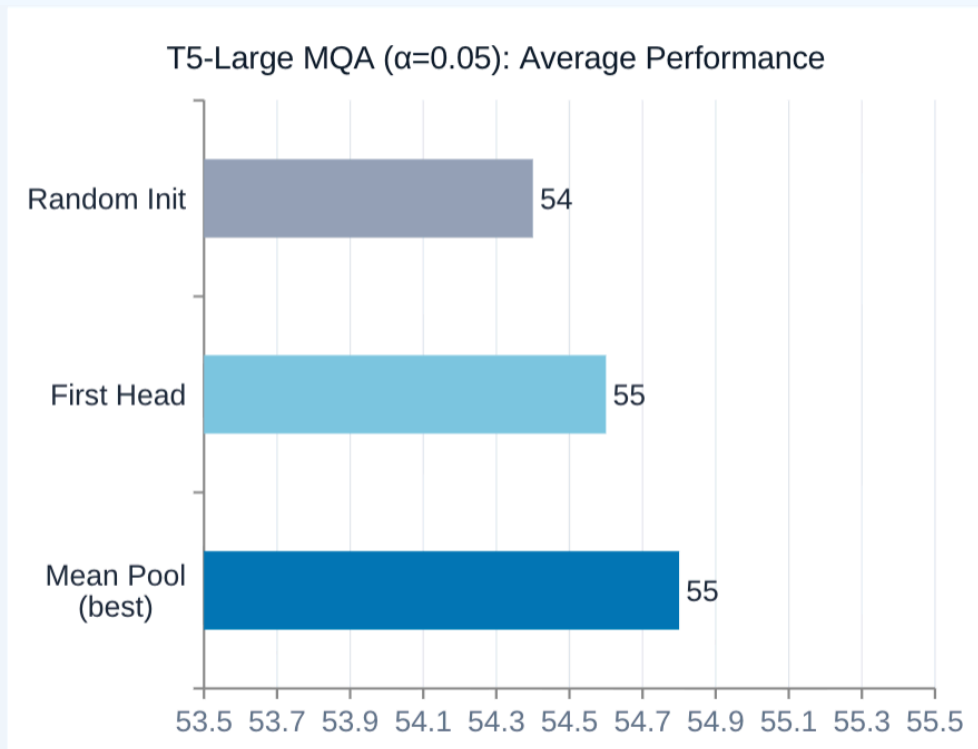


Key Insight

Q projections are independent per head (that's fine — queries are computed fresh each step).
But K,V are the cache — we want fewer of them. Mean-pooling merges the "knowledge" from all K heads into one.

Fig 4: How You Initialize the New Heads Matters

6 / 12



Why the ordering?

More preserved info → better start

Mean: aggregates signal from all original K/V heads

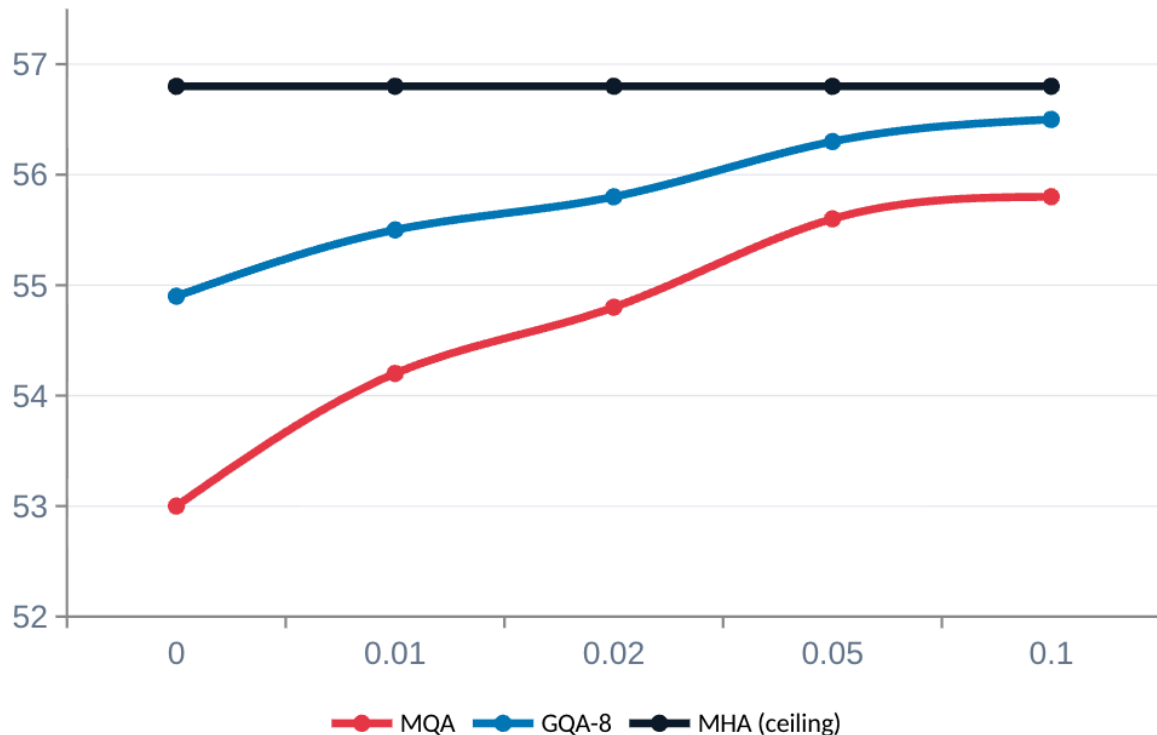
First: keeps one, discards the rest

Random: starts blind — must re-learn everything from $\alpha=5\%$ data

Fig 5: How Much Uptraining Do You Need?

7 / 12

Performance vs Uptraining Proportion α



Takeaways

GQA works even at $\alpha=0$
(mean pool is already reasonable)

MQA needs uptraining
(pooling to 1 head is lossy)

Both plateau by $\alpha=5-10\%$
(diminishing returns past 5%)

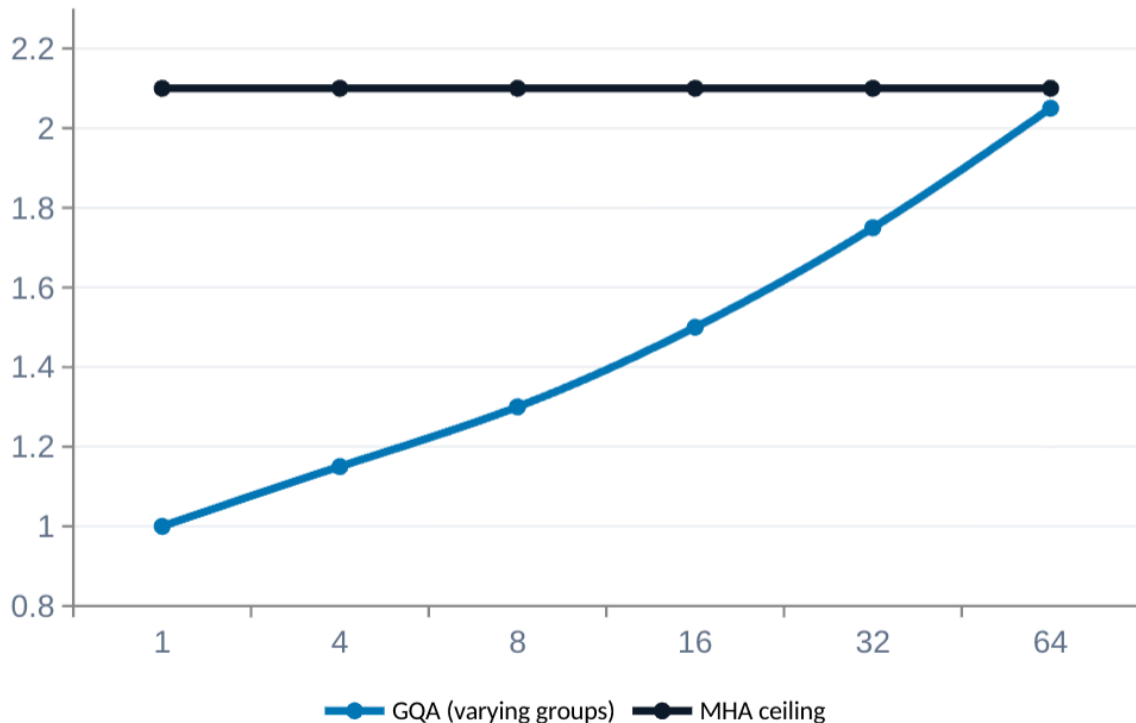
Critical threshold: $\alpha=0.05$
Sweet spot: big gains, low cost



Q: Why does GQA need less uptraining than MQA? What does this tell us about capacity vs bandwidth?

Fig 6: More Groups = Slower — How Many Is Enough?

Inference Time vs Number of GQA Groups (T5-XXL)



Why G=8?

G=1 (MQA): fastest but capacity-limited

**G=8: modest overhead,
large quality gain**

G>16: cost rises steeply, quality gains shrink

Sharding note:

MQA wastes shards (single KV must be replicated). GQA-G aligns better with tensor parallelism.

Main Results: Quality vs Speed (Fig 3 + Table 1)

9 / 12

Avg Performance vs Inference Time (ms)

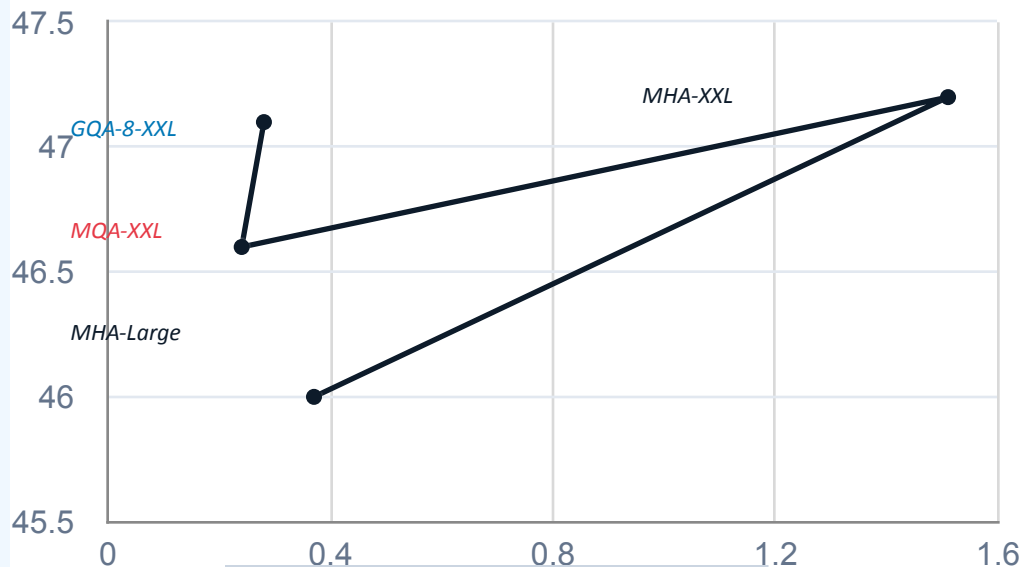


Table 1 summary

Model	Time(s)	Avg
MHA-Large	0.37	46.0
MHA-XXL	1.51	47.2
MQA-XXL	0.24 ✓	46.6
GQA-8-XXL	0.28 ✓	47.1 ✓

What This Means

GQA-8-XXL hits the sweet spot:

Speed: 6× faster than MHA-XXL, comparable to MQA

Quality: Only 0.1 pts below MHA-XXL on average

Practical win: Serves bigger models at same latency budget

Tradeoff to discuss: We don't have a GQA-8-XXL trained from scratch to compare against!



What Improved

- 6× faster decoder inference
- KV-cache size divided by H/G
- Stable training (vs MQA instability)
- Works with existing checkpoints



What Stayed the Same

- Q-projection compute (recomputed every step)
- Encoder attention (not the bottleneck)
- Total parameter count (mostly)



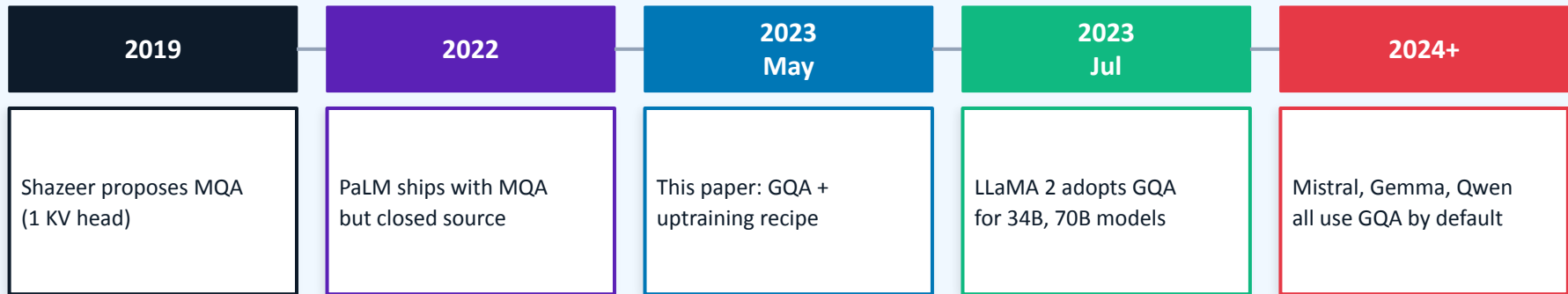
What Didn't Get Addressed

- No scratch training comparison for GQA
- Only tested on T5 (enc-dec), not decoder-only
- ROUGE is flawed — may miss quality degradation
- Long-seq quality hard to eval rigorously



Questions

- You're designing a new 70B model from scratch. Do you use MHA, MQA or GQA? What trade-offs would you consider?
- GQA is never compared to a model trained from scratch with GQA. Does this matter? In what scenarios would uptraining be clearly suboptimal?



The Broader Lesson

You don't need to retrain from scratch. The uptraining recipe lets organizations convert their existing expensive MHA checkpoints.

GQA solved a practical gap: MQA was known to be fast but unstable; many orgs avoided it. GQA gives most of the speed with almost none of the instability.

Today, GQA is the default in virtually every major open-source LLM. This paper is why.

MQA: Training Instability

Observed:

- Frequent loss spikes during pre-training
- Models diverge immediately on long-input fine-tuning
- Required 3-run averages to get stable metrics

Root cause (hypothesis):

Single K,V head must simultaneously serve all Q heads — gradient conflicts when heads want to encode different information in the same projection.

Practical consequence:

Many teams shipped MHA or avoided MQA entirely despite the speed benefits.

GQA: Stability Restored

Observed:

- GQA uptrained models: stable across all tasks
- Single fine-tuning run sufficient
- No divergence even on long inputs (2048 tokens)

Why more stable?

Groups allow K,V heads to specialize. G groups → G sub-problems, each with less conflicting gradient pressure.

Unstated implication:

Stability alone might justify GQA over MQA even if speed gains were identical.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. [arXiv:2305.13245](#).

Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. [arXiv:1911.02150](#).

Pope, R., Douglas, S., Chowdhery, A., et al. (2022). Efficiently Scaling Transformer Inference. [arXiv:2211.05102](#).

Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with T5. *JMLR* 21, 140:1–67.

Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. [arXiv:2302.13971](#).

Dao, T., et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention. [arXiv:2205.14135](#).

Komatsuzaki, A., et al. (2022). Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints. [arXiv:2212.09596](#).

Chowdhery, A., et al. (2022). PaLM: Scaling Language Modeling with Pathways. [arXiv:2204.02311](#).